

## CS4423 - Networks

Prof. Götz Pfeiffer

School of Mathematics, Statistics and Applied Mathematics

NUI Galway

### 5. Power Laws and Scale-Free Graphs

## Lecture 15: Hubs and Authorities

We'll now look at centrality measures in directed networks.

```
In [1]: import networkx as nx
```

### In-Degree vs. Out-Degree

In a directed network, one can distinguish between **in-degree centrality**  $c_i^{D^{\text{in}}}$  and **out-degree centrality**  $c_i^{D^{\text{out}}}$  based on the number of arrows that point **into** a node, and the number of arrows pointing **out** of a node:

$$c_i^{D^{\text{in}}} = k_i^{\text{in}} = \sum_{j=1}^n a_{ij}, \quad c_i^{D^{\text{out}}} = k_i^{\text{out}} = \sum_{j=1}^n a_{ji},$$

where  $A = (a_{ij})$  is the adjacency matrix of a directed graph  $G = (X, E)$  ...

There are also the corresponding **eigenvector centralities**  $c^{E^{\text{in}}}$  and  $c^{E^{\text{out}}}$ :

$$Ac^{E^{\text{in}}} = \lambda c^{E^{\text{in}}}, \quad A^T c^{E^{\text{out}}} = \lambda c^{E^{\text{out}}}.$$

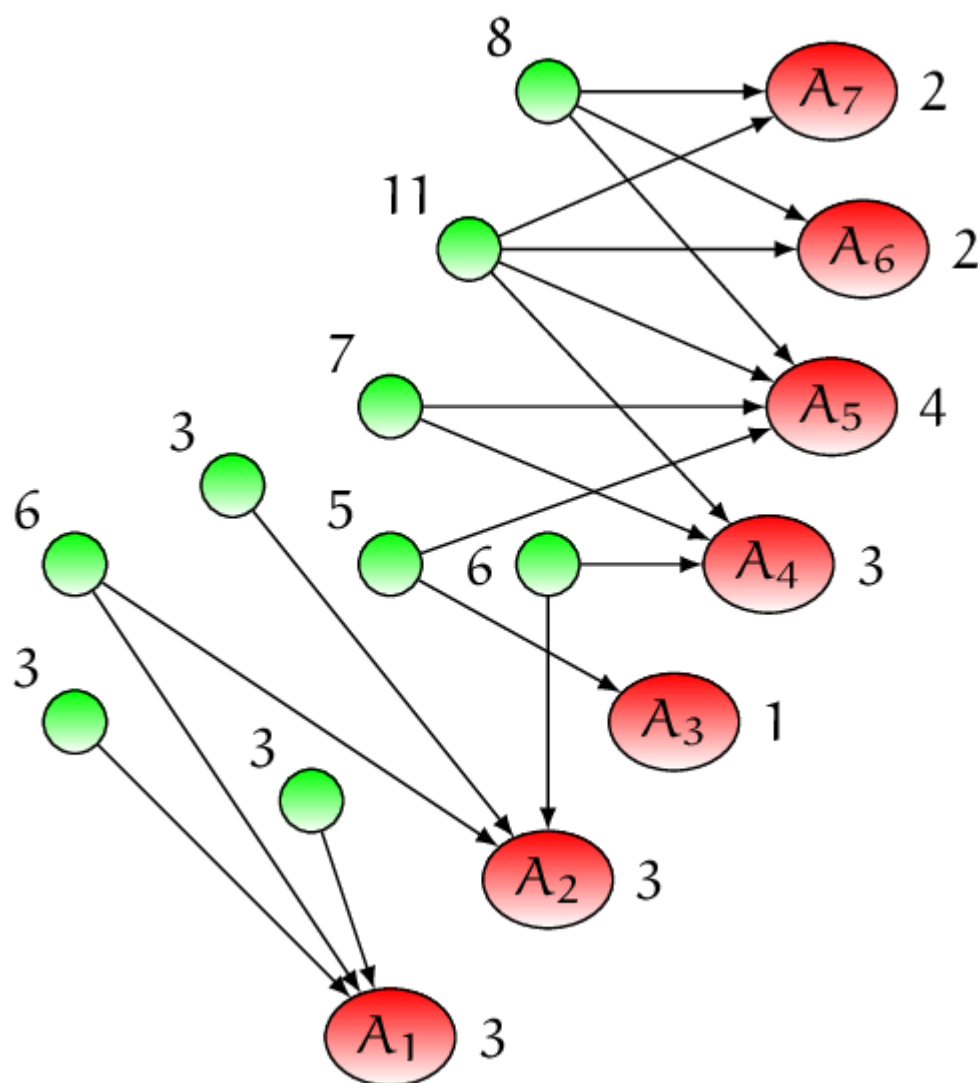
Here, we take a slightly different approach.

## Hub Centrality and Authority Centrality

In a network of nodes connected by directed edges, each node plays two different roles, one as a receiver of links, and one as a sender of links. A first measure of importance, or recognition, of a node in this network might be the number of links it receives, i.e., its **in-degree** in the underlying graph. If in a collection of web pages relating to a search query on the subject of "networks", say, a particular page receives a high number of links, this page might count as an **authority** on that subject, with **authority score** measured by its in-degree.

In turn, the web pages linking to an authority in some sense know where to find valuable information and thus serve as good "lists" for the subject. A high value list is called a **hub** for this query. It makes sense to measure the value of a page as list in terms of the values of the pages

it points at, by assigning to its **hub score** the sum of the authority scores of the pages it points at.



Now the authority score of a page could take the hub scores of the list pages into account, by using the sum of the hub scores of the pages that point at it as an updated authority score.

Then again, applying the **Principle of Repeated Improvement**, the hub scores can be updated on the basis of the new authority scores, and so on.

This suggests a ranking procedure that tries to estimate, for each page  $p$ , its value as an authority and its value as a hub in the form of numerical scores,  $a(p)$  and  $h(p)$ .

Starting off with values all equal to 1, the estimates are updated by applying the following two rules in an alternating fashion.

**Authority Update Rule:** For each page  $p$ , update  $a(p)$  to be the sum of the hub scores of all the pages pointing to it.

**Hub Update Rule:** For each page  $p$ , update  $h(p)$  to be the sum of the authority scores of all the pages that it points to.

In order to keep the numbers from growing too large, score vectors should be **normalized** after each step, in a way that replaces  $h$  by a scalar multiple  $\hat{h} = sh$  so that the entries in  $\hat{h}$  add up to 100, say, representing relative percentage values, similarly for  $a$ .

After a number of iterations, the values  $a(p)$  and  $h(p)$  stabilize, in the sense that further applications of the update rules do not yield essentially better relative estimates.

**Example.** Continuing the example above ...

```
In [2]: nodes = list(range(1,10)) + ["A%s" % (i+1) for i in range(7)]
        print(nodes)
```

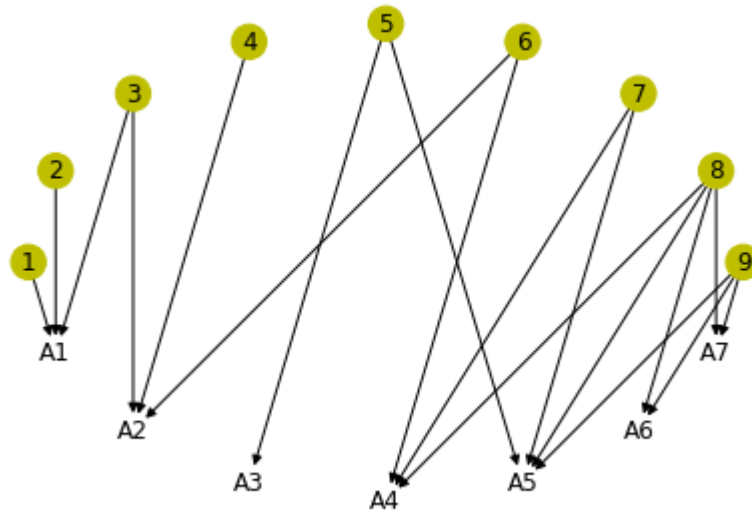
```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 'A1', 'A2', 'A3', 'A4', 'A5', 'A6', 'A7']
```

```
In [3]: edges = [
        (1, "A1"), (2, "A1"), (3, "A1"), (3, "A2"), (4, "A2"), (5, "A3"),
        (5, "A5"), (6, "A2"), (6, "A4"), (7, "A4"), (7, "A5"), (8, "A4"),
        (8, "A5"), (8, "A6"), (8, "A7"), (9, "A5"), (9, "A6"), (9, "A7")
        ]
```

```
In [4]: G = nx.DiGraph()
        G.add_nodes_from(nodes)
        G.add_edges_from(edges)
```

```
In [5]: pos = nx.circular_layout(G)
        for i in [1,2,3,4]:
            j = 10 - i
            pos[i], pos[j] = pos[j], pos[i]
        colors = 9 * ['y'] + 7 * ['w']
```

```
In [6]: nx.draw(G, with_labels=True, node_color=colors, pos=pos)
```



Let's use dictionaries, with nodes as keys and hub or authority scores as values. Here's a way to normalize such a record.

```
In [7]: def normalized(d):
         s = sum(d.values())
         return { k: 100/s*v for k, v in d.items() }
```

Initially, all scores are set to 1 (and then normalized).

```
In [8]: hubs = normalized({ x : 1 for x in G })
         auth = normalized({ x : 1 for x in G })
```

The update rules can then be implemented as follows.

```
In [9]: def HubsUpdate(G, auth):  
        h = { x: 0 for x in G }  
        for x in G:  
            for y in G.successors(x):  
                h[x] += auth[y]  
        return normalized(h)  
  
        def AuthUpdate(G, hubs):  
            a = { x: 0 for x in G }  
            for x in G:  
                for y in G.successors(x):  
                    a[y] += hubs[x]  
            return normalized(a)
```

Now we can apply the rules. alternating between the two, say 10 times, and observe how the scores stabilize.

```
In [10]: for k in range(10):
          auth = AuthUpdate(G, hubs)
          print("auth= ", auth)
          hubs = HubsUpdate(G, auth)
          print("hubs = ", hubs)
```

```
auth= {1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0, 5: 0.0, 6: 0.0, 7: 0.0, 8: 0.0, 9: 0.0, 'A1': 16.666666666666664, 'A2': 16.666666666666664, 'A3': 5.555555555555555, 'A4': 16.666666666666664, 'A5': 22.22222222222222, 'A6': 11.111111111111111, 'A7': 11.111111111111111}
hubs = {1: 5.769230769230769, 2: 5.769230769230769, 3: 11.538461538461538, 4: 5.769230769230769, 5: 9.615384615384617, 6: 11.538461538461538, 7: 13.461538461538462, 8: 21.153846153846157, 9: 15.384615384615387, 'A1': 0.0, 'A2': 0.0, 'A3': 0.0, 'A4': 0.0, 'A5': 0.0, 'A6': 0.0, 'A7': 0.0}
auth= {1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0, 5: 0.0, 6: 0.0, 7: 0.0, 8: 0.0, 9: 0.0, 'A1': 9.599999999999998, 'A2': 11.999999999999998, 'A3': 4.0, 'A4': 19.2, 'A5': 24.8, 'A6': 15.200000000000001, 'A7': 15.200000000000001}
hubs = {1: 3.3519553072625694, 2: 3.3519553072625694, 3: 7.541899441340782, 4: 4.189944134078212, 5: 10.055865921787712, 6: 10.893854748603353, 7: 15.363128491620113, 8: 25.977653631284923, 9: 19.27374301675978, 'A1': 0.0, 'A2': 0.0, 'A3': 0.0, 'A4': 0.0, 'A5': 0.0, 'A6': 0.0, 'A7': 0.0}
auth= {1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0, 5: 0.0, 6: 0.0, 7: 0.0, 8: 0.0, 9: 0.0, 'A1': 5.472103004291844, 'A2': 8.690987124463517, 'A3': 3.862660944206009, 'A4': 20.064377682403432, 'A5': 27.145922746781117, 'A6': 17.38197424892704, 'A7': 17.38197424892704}
hubs = {1: 1.9223520542781751, 2: 1.9223520542781751, 3: 4.975499434602336, 4: 3.05314738032416, 5: 10.89332830757633, 6: 10.101771579344138, 7: 16.584998115341122, 8: 28.79758763663777, 9: 21.748963437617792, 'A1': 0.0, 'A2': 0.0, 'A3': 0.0, 'A4': 0.0, 'A5': 0.0, 'A6': 0.0, 'A7': 0.0}
auth= {1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0, 5: 0.0, 6: 0.0, 7: 0.0, 8: 0.0, 9: 0.0, 'A1': 3.237410071942446, 'A2': 6.654676258992806, 'A3': 3.9983397897066975, 'A4': 20.365246264526842, 'A5': 28.638627559490875, 'A6': 18.552850027670175, 'A7': 18.552850027670175}
hubs = {1: 1.1417975992973552, 2: 1.1417975992973552, 3: 3.48882599785303, 4: 2.347028398555675, 5: 11.510686054454963, 6: 9.52961842490485, 7: 17.283107250902706, 8: 30.369864350541626, 9: 23.187274324192447, 'A1': 0.0, 'A2': 0.0, 'A3': 0.0, 'A4': 0.0, 'A5': 0.0, 'A6': 0.0, 'A7': 0.0}
auth= {1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0, 5: 0.0, 6: 0.0, 7: 0.0, 8: 0.0, 9: 0.0, 'A1': 2.0667726550079486, 'A2': 5.501493736787854, 'A3': 4.121315886021768, 'A4': 20.473802826744002, 'A5': 29.485141249847132, 'A6': 19.17573682279565, 'A7': 19.17573682279565}
hubs = {1: 0.7305896593463598, 2: 0.7305896593463598, 3: 2.675329166769595, 4: 1.9447395074232352, 5: 11.87964724191596, 6: 9.182085422790937, 7: 17.66013685432672, 8: 31.21711420172426, 9: 23.979768286356563, 'A1': 0.0, 'A2': 0.0, 'A3': 0.0, 'A4': 0.0, 'A5': 0.0, 'A6': 0.0, 'A7': 0.0}
auth= {1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0, 5: 0.0, 6: 0.0, 7: 0.0, 8: 0.0, 9: 0.0, 'A1': 1.4616220556978818, 'A2': 4.8769470472965, 'A3': 4.197635393162804, 'A4': 20.515081024226525, 'A5': 29.941430373283172, 'A6': 19.50364205316656, 'A7': 19.50364205316656}
hubs = {1: 0.5173171490866625, 2: 0.5173171490866625, 3: 2.243432551
```

```

4362925, 4: 1.7261154023496297, 5: 12.082962285587177, 6: 8.987091786
335196, 7: 17.858254483273694, 8: 31.664242788415123, 9: 24.403266404
429555, 'A1': 0.0, 'A2': 0.0, 'A3': 0.0, 'A4': 0.0, 'A5': 0.0, 'A6':
0.0, 'A7': 0.0}
auth= {1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0, 5: 0.0, 6: 0.0, 7: 0.0, 8: 0.
0, 9: 0.0, 'A1': 1.1503159350860046, 'A2': 4.546651987894891, 'A3':
4.2400673011928935, 'A4': 20.53176940451779, 'A5': 30.18157120315517
6, 'A6': 19.674812084076624, 'A7': 19.674812084076624}
hubs = {1: 0.4074051889999927, 2: 0.4074051889999927, 3: 2.017684205
3530253, 4: 1.6102790163530327, 5: 12.191045705634656, 6: 8.881976416
708037, 7: 17.961046600250107, 8: 31.897427539028076, 9: 24.625730138
673077, 'A1': 0.0, 'A2': 0.0, 'A3': 0.0, 'A4': 0.0, 'A5': 0.0, 'A6':
0.0, 'A7': 0.0}
auth= {1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0, 5: 0.0, 6: 0.0, 7: 0.0, 8: 0.
0, 9: 0.0, 'A1': 0.9903983191092612, 'A2': 4.374173656980833, 'A3':
4.262670525834496, 'A4': 20.538942540713393, 'A5': 30.30650875614401
2, 'A6': 19.76365310060901, 'A7': 19.76365310060901}
hubs = {1: 0.35088920088975745, 2: 0.35088920088975745, 3: 1.9006195
158921473, 4: 1.5497303150023896, 5: 12.24754874844453, 6: 8.82649259
9608509, 7: 18.014085272760987, 8: 32.01825371555903, 9: 24.741491430
95291, 'A1': 0.0, 'A2': 0.0, 'A3': 0.0, 'A4': 0.0, 'A5': 0.0, 'A6':
0.0, 'A7': 0.0}
auth= {1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0, 5: 0.0, 6: 0.0, 7: 0.0, 8: 0.
0, 9: 0.0, 'A1': 0.9082573539193753, 'A2': 4.284714625959304, 'A3':
4.274490900382077, 'A4': 20.542195438223864, 'A5': 30.37114618044269,
'A6': 19.809597750536348, 'A7': 19.809597750536348}
hubs = {1: 0.32184542832009577, 2: 0.32184542832009577, 3: 1.8401549
779983077, 4: 1.518309549678212, 5: 12.276850671875161, 6: 8.79753680
9353295, 7: 18.041391208952078, 8: 32.08064659258893, 9: 24.801419332
91385, 'A1': 0.0, 'A2': 0.0, 'A3': 0.0, 'A4': 0.0, 'A5': 0.0, 'A6':
0.0, 'A7': 0.0}
auth= {1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0, 5: 0.0, 6: 0.0, 7: 0.0, 8: 0.
0, 9: 0.0, 'A1': 0.8660528826269034, 'A2': 4.23848366606995, 'A3': 4.
280620707486448, 'A4': 20.54373372264567, 'A5': 30.40449487180698, 'A
6': 19.833307074682025, 'A7': 19.833307074682025}
hubs = {1: 0.30691870338820654, 2: 0.30691870338820654, 3: 1.8089862
297689223, 4: 1.502067526380716, 5: 12.291986914444484, 6: 8.78251915
1669392, 7: 18.0554381537366, 8: 32.11280812125607, 9: 24.83235649596
74, 'A1': 0.0, 'A2': 0.0, 'A3': 0.0, 'A4': 0.0, 'A5': 0.0, 'A6': 0.0,
'A7': 0.0}

```

All in one python function:

```

In [11]: def HubsAuth(G, k):
          hubs = normalized({ x : 1 for x in G })
          auth = normalized({ x : 1 for x in G })
          for i in range(k):
              auth = AuthUpdate(G, hubs)
              hubs = HubsUpdate(G, auth)

          return hubs, auth

```

```
In [12]: hubs, auth = HubsAuth(G, 10)
hubs
```

```
Out[12]: {1: 0.30691870338820654,
2: 0.30691870338820654,
3: 1.8089862297689223,
4: 1.502067526380716,
5: 12.291986914444484,
6: 8.782519151669392,
7: 18.0554381537366,
8: 32.11280812125607,
9: 24.8323564959674,
'A1': 0.0,
'A2': 0.0,
'A3': 0.0,
'A4': 0.0,
'A5': 0.0,
'A6': 0.0,
'A7': 0.0}
```

```
In [13]: auth
```

```
Out[13]: {1: 0.0,
2: 0.0,
3: 0.0,
4: 0.0,
5: 0.0,
6: 0.0,
7: 0.0,
8: 0.0,
9: 0.0,
'A1': 0.8660528826269034,
'A2': 4.23848366606995,
'A3': 4.280620707486448,
'A4': 20.54373372264567,
'A5': 30.40449487180698,
'A6': 19.833307074682025,
'A7': 19.833307074682025}
```

Finally, let's apply this to a random directed graph.

```
In [14]: n, m = 80, 120
G = nx.gnm_random_graph(n, m, directed=True)
```

```
In [15]: hubs, auth = HubsAuth(G, 50)
```

Let's inspect the top and the bottom 10 scores.



```
In [16]: [(k,auth[k]) for k in sorted(auth, key=auth.get, reverse=True)][:10]
```

```
Out[16]: [(9, 18.37082210566947),
(41, 11.997949229720508),
(79, 9.851532424256343),
(23, 8.538373504013734),
(11, 8.158244902032731),
(32, 5.83529403089815),
(38, 5.83529403089815),
(76, 5.737936878874051),
(48, 5.705992436271492),
(7, 2.5481224730873033)]
```

```
In [17]: [(k,auth[k]) for k in sorted(auth, key=auth.get)][:10]
```

```
Out[17]: [(2, 0.0),
(3, 0.0),
(26, 0.0),
(33, 0.0),
(43, 0.0),
(45, 0.0),
(47, 0.0),
(56, 0.0),
(60, 0.0),
(68, 0.0)]
```

```
In [18]: [(k, hubs[k]) for k in sorted(hubs, key=hubs.get, reverse=True)][:10]
```

```
Out[18]: [(43, 17.173546109723112),
(29, 16.887018747185234),
(54, 13.25540759716817),
(8, 6.750223963961408),
(67, 5.1681036900138215),
(1, 4.283169654923388),
(26, 4.243708140522168),
(62, 4.242923689642151),
(34, 3.8388669738677517),
(72, 3.6721211819336634)]
```

```
In [19]: [(k, hubs[k]) for k in sorted(hubs, key=hubs.get)][:10]
```

```
Out[19]: [(5, 0.0),
(6, 0.0),
(11, 0.0),
(16, 0.0),
(20, 0.0),
(30, 0.0),
(31, 0.0),
(32, 0.0),
(38, 0.0),
(39, 0.0)]
```

In terms of matrix algebra this process can be described as follows.

## Spectral Analysis of Hubs and Authorities

Let  $M = (m_{ij})$  be the **adjacency matrix** of the directed graph  $G = (X, E)$  that is  $m_{ij} = 1$  if  $x_j \rightarrow x_i$  and  $m_{ij} = 0$  otherwise, where  $X = \{x_1, \dots, x_n\}$ .

We write  $h = (h_1, \dots, h_n)$  for a list of hub scores, with  $h_i = h(x_i)$ , the hub score of node  $x_i$ . Similarly, we write  $a = (a_1, \dots, a_l)$  for a list of authority scores.

The **hub update rule** can now be expressed as a **matrix multiplication**:

$$h \leftarrow M^T a$$

and similarly, the **authority update rule**, using the transpose of the matrix  $M$ :

$$a \leftarrow M h$$

Applying two steps of the procedure at once yields update rules

$$h \leftarrow M^T M h$$

and

$$a \leftarrow M M^T a$$

for  $h$  and  $a$ , respectively.

**In the limit**, one expects to get vectors  $h^*$  and  $a^*$  whose directions do not change under the latter rules, i.e.,

$$(M^T M)h^* = c h^*$$

and

$$(M M^T)a^* = d a^*$$

for certain constants  $c$  and  $d$ , meaning that  $h^*$  and  $a^*$  are **eigenvectors** for the matrices  $M^T M$  and  $M M^T$ , respectively.

Using the fact that  $M^T M$  and  $M M^T$  are **symmetric** matrices ( $(M^T M)^T = M^T (M^T)^T = M^T M$ ), it can indeed be shown that any sequence of hub score vectors  $h$  under repeated application of the above update rule converges to a real-valued eigenvector  $h^*$  of  $M M^T$  for the real eigenvalue  $c$ . The argument uses the [Spectral Theorem](https://en.wikipedia.org/wiki/Spectral_theorem) ([https://en.wikipedia.org/wiki/Spectral\\_theorem](https://en.wikipedia.org/wiki/Spectral_theorem)) for [real symmetric matrices](https://en.wikipedia.org/wiki/Symmetric_matrix#Real_symmetric_matrices) ([https://en.wikipedia.org/wiki/Symmetric\\_matrix#Real\\_symmetric\\_matrices](https://en.wikipedia.org/wiki/Symmetric_matrix#Real_symmetric_matrices)).

A similar result exists for any sequence of authority score vectors  $a$ .

## PageRank

A simpler model of endorsement for web pages involves only one numerical value  $r(p)$  per page  $p$ , built on the principle that **a page is as important as the pages linking to it**. As before, these importance values can be obtained by repeatedly applying a suitable update rule to a set of current values.

Specifically, PageRank is computed as follows.

- If the network has  $n$  nodes, each page  $p$  receives an initial PageRank of  $r(p) = 1/n$ .
- Choose a number of steps,  $k$ .

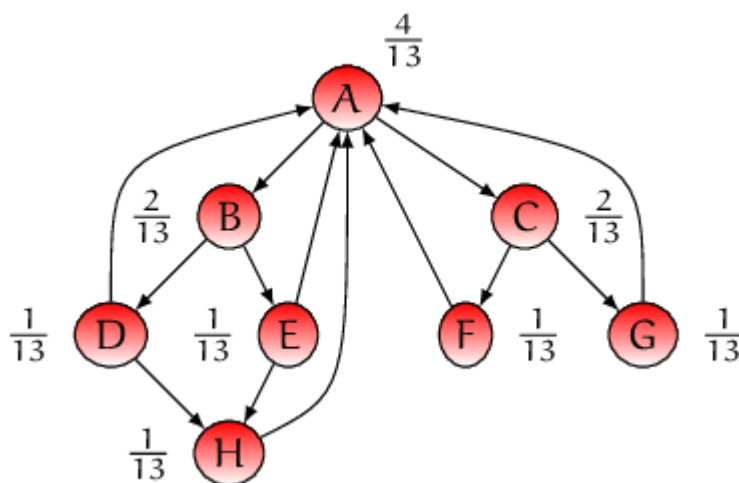
- Perform the following update rule  $k$  times.

**Basic PageRank Update Rule:** Each page divides its current PageRank by the number of pages it links to, and passes this value on to those pages. In this way, each page updates its PageRank to be the sum of all the shares it receives from the pages linking to it.

As in each step, the total PageRank of all nodes is maintained (each node splits its PageRank into equal parts and passes this on, nothing is lost or gained overall), there is no need for normalization.

After a number of steps, the PageRank values of the individual nodes stabilize. These values form an equilibrium in the sense that another application of the update rule will produce exactly the same values.

**Example.** The following graph represents a network of 8 web pages with hyperlinks.



The following table shows how the initial PageRank of  $\frac{1}{8}$  of each page is updated under six iterations of the basic PageRank update rule and, in the bottom row, the limit values.

|          | A                | B                | C                | D                | E                | F                | G                | H                |
|----------|------------------|------------------|------------------|------------------|------------------|------------------|------------------|------------------|
| 1        | $\frac{1}{8}$    | $\frac{1}{8}$    | $\frac{1}{8}$    | $\frac{1}{8}$    | $\frac{1}{8}$    | $\frac{1}{8}$    | $\frac{1}{8}$    | $\frac{1}{8}$    |
| 2        | $\frac{8}{16}$   | $\frac{1}{16}$   | $\frac{1}{16}$   | $\frac{1}{16}$   | $\frac{1}{16}$   | $\frac{1}{16}$   | $\frac{1}{16}$   | $\frac{2}{16}$   |
| 3        | $\frac{10}{32}$  | $\frac{8}{32}$   | $\frac{8}{32}$   | $\frac{1}{32}$   | $\frac{1}{32}$   | $\frac{1}{32}$   | $\frac{1}{32}$   | $\frac{2}{32}$   |
| 4        | $\frac{5}{32}$   | $\frac{5}{32}$   | $\frac{5}{32}$   | $\frac{4}{32}$   | $\frac{4}{32}$   | $\frac{4}{32}$   | $\frac{4}{32}$   | $\frac{1}{32}$   |
| 5        | $\frac{26}{64}$  | $\frac{5}{64}$   | $\frac{5}{64}$   | $\frac{5}{64}$   | $\frac{5}{64}$   | $\frac{5}{64}$   | $\frac{5}{64}$   | $\frac{8}{64}$   |
| 6        | $\frac{46}{128}$ | $\frac{26}{128}$ | $\frac{26}{128}$ | $\frac{5}{128}$  | $\frac{5}{128}$  | $\frac{5}{128}$  | $\frac{5}{128}$  | $\frac{10}{128}$ |
| 7        | $\frac{25}{128}$ | $\frac{23}{128}$ | $\frac{23}{128}$ | $\frac{13}{128}$ | $\frac{13}{128}$ | $\frac{13}{128}$ | $\frac{13}{128}$ | $\frac{5}{128}$  |
| $\infty$ | $\frac{4}{13}$   | $\frac{2}{13}$   | $\frac{2}{13}$   | $\frac{1}{13}$   | $\frac{1}{13}$   | $\frac{1}{13}$   | $\frac{1}{13}$   | $\frac{1}{13}$   |

For a slightly larger example, let's implement this algorithm as a python program.

```
In [20]: n, m = 80, 120
G = nx.gnm_random_graph(n, m, directed=True)
```

```
In [21]: G.out_degree()
```

```
Out[21]: OutDegreeView({0: 3, 1: 0, 2: 0, 3: 2, 4: 0, 5: 0, 6: 1, 7: 3, 8: 3,
9: 2, 10: 1, 11: 2, 12: 0, 13: 1, 14: 1, 15: 0, 16: 1, 17: 1, 18: 3, 1
9: 3, 20: 4, 21: 2, 22: 1, 23: 0, 24: 0, 25: 3, 26: 0, 27: 0, 28: 3, 2
9: 0, 30: 2, 31: 2, 32: 2, 33: 2, 34: 2, 35: 0, 36: 0, 37: 3, 38: 1, 3
9: 4, 40: 2, 41: 4, 42: 1, 43: 1, 44: 2, 45: 2, 46: 1, 47: 2, 48: 2, 4
9: 1, 50: 1, 51: 4, 52: 4, 53: 2, 54: 4, 55: 1, 56: 1, 57: 1, 58: 1, 5
9: 2, 60: 2, 61: 0, 62: 1, 63: 0, 64: 0, 65: 2, 66: 1, 67: 2, 68: 2, 6
9: 0, 70: 1, 71: 2, 72: 1, 73: 0, 74: 2, 75: 2, 76: 0, 77: 1, 78: 2, 7
9: 2})
```

The simple algorithm doesn't work if there is a node  $x$  with no successors in  $G$ . (Why?) So for now, let's add some random edges to make sure each node  $x$  has out-degree at least 1.

```
In [22]: import random

for x in G:
    y = x
    while y == x:
        y = random.randrange(n)
    G.add_edge(x, y)
```

```
In [23]: def PageRank(G, k):
n = G.order()
r = { x: 1/n for x in G }
for i in range(k):
    s = { x : 0 for x in G }
    for x in G:
        l = G.out_degree(x)
        v = r[x]/l
        for y in G.successors(x):
            s[y] += v
    r = s
return r
```

```
In [24]: k = 50
pr = PageRank(G, k)
[(k,pr[k]) for k in sorted(pr, key=pr.get)][:10]
```

```
Out[24]: [(19, 0),
(20, 0),
(28, 0.0),
(30, 0),
(36, 0),
(40, 0.0),
(43, 0),
(51, 0),
(58, 0),
(67, 0)]
```

```
In [25]: [(k,pr[k]) for k in sorted(pr, key=pr.get, reverse=True)][:10]
```

```
Out[25]: [(38, 0.04548183374988097),
(0, 0.04415149420348832),
(62, 0.04198361309636439),
(11, 0.041922285888588304),
(8, 0.03900435068112444),
(10, 0.0315396768795158),
(37, 0.03125797537645103),
(59, 0.02845378563309569),
(41, 0.028394958095606947),
(29, 0.028394956574366283)]
```

In terms of matrix algebra this process can be described as follows.

## Spectral Analysis of PageRank

Here, we use a **variant of the adjacency matrix**  $A$  of the directed graph  $G = (X, V)$ .

Let  $N$  be the  $n \times n$  matrix with entries  $N_{ij} = 0$  if node  $x_j$  is not linked to node  $x_i$  (as for the adjacency matrix  $A$ ). And when  $x_j \rightarrow x_i$ , then set  $N_{ij} = 1/l_j$ , where  $l_j$  is the number of links out of  $x_j$ .

Write  $r = (r_1, \dots, r_n)$  for the list of PageRank values of the nodes  $X = \{x_1, \dots, x_n\}$ . Then the **basic PageRank update rule** can be expressed as **matrix multiplication**:

$$r \leftarrow N r.$$

It can be shown that repeated application of the basic PageRank update rule lets the PageRank values converge towards a vector  $r^*$  with the property

$$N r^* = r^*,$$

that is,  $r^*$  is an **eigenvector** of  $N$  for the eigenvalue 1.

The argument uses the [Perron-Frobenius Theorem](https://en.wikipedia.org/wiki/Perron-Frobenius_Theorem) ([https://en.wikipedia.org/wiki/Perron%E2%80%93Frobenius\\_theorem](https://en.wikipedia.org/wiki/Perron%E2%80%93Frobenius_theorem)) which we have seen before (in week 8). Recall that, for a matrix in which all entries are non-negative (such as the matrix  $N^T$ ) the theorem guarantees the existence of a **real eigenvalue** with corresponding **eigenvector having non-negative entries**. (Not every matrix with real entries has this property.)

## Code Corner

### python

- sorted : [\[doc\]](https://docs.python.org/3/library/functions.html#sorted) (<https://docs.python.org/3/library/functions.html#sorted>)

### random

- randrange vs. randint : [\[doc\]](https://docs.python.org/3/library/random.html) (<https://docs.python.org/3/library/random.html>)

### networkx

- DiGraph : [\[doc\]](https://networkx.github.io/documentation/stable/reference/classes/digraph.html) (<https://networkx.github.io/documentation/stable/reference/classes/digraph.html>)
- circular\_layout : [\[doc\]](https://networkx.github.io/documentation/stable/reference/generated/networkx.drawing.layout) (<https://networkx.github.io/documentation/stable/reference/generated/networkx.drawing.layout>)



## Exercises

1. It might be tempting to combine the hubs and authorities update rules into a single for loop. Why is this not a good idea?
2. Create a random  $G(n, m)$  graph (with  $n = 80$  and  $m = 100$ , say) and identify its Bow-Tie components (Giant SCC, IN, OUT, tendrils and tubes). Then compute Hubs and Authority centralities to identify strong hubs and heavy authorities. Which Bow-Tie components do

hubs and authorities prefer?

3. Create a random  $G(n, m)$  graph (with  $n = 80$  and  $m = 100$ , say) and complete the graph with further random edges so that each node has out-degree at least 1. Identify the graph's Bow-Tie components (Giant SCC, IN, OUT, tendrils and tubes). Then compute PageRank centralities to identify important pages. Which Bow-Tie components do these pages prefer?

In [ ]: